NEXUS CPU ANALYZER

Complete Technology Stack & Commercial Product Architecture

Official Technical Documentation • Updated 13 November 2026, 06:37 PM

This master document provides an exhaustive technical breakdown of every library, compiler pipeline, design pattern, and architectural decision powering the NEXUS CPU Analyzer. It details the product's evolution from an internal Python script into a **Dual-Surface Commercial Product** featuring a public Vercel distribution showcase and an autonomous, zero-dependency 11.7 MB desktop executable (.EXE).

1. Commercial Product Architecture Overview

Attribute	Specification / Architecture
Product Name	NEXUS CPU Analyzer (NEXUS PRO Enterprise Edition)
Primary Delivery	Standalone Desktop Executable (NEXUS_CPU_Analyzer.exe — 11.7 MB)
Distribution Surface	Vercel Cloud Landing Page (Public Showcase, Docs & Direct Binary Download)
Target Platforms	Windows 10 / Windows 11 (64-bit architecture)
Local Runtime Port	http://127.0.0.1:8050 (Private local loopback)
Execution Model	100% Client-Side Local Execution — Zero Cloud Telemetry Relay (Absolute Privacy)
Core Languages	Python 3.11.9 (Backend Telemetry) | Vanilla JavaScript ES6+ (Frontend SPA)
Compiler & Packaging	PyInstaller 6.x (Custom spec build, embedded Python DLLs & resource bundling)
Core Features	Sub-millisecond Kernel Sampler, Smart Countermeasures (BSOD Guard), Hardware Digital Twin, Z-Score Anoma

2. Standalone Binary Packaging Pipeline (PyInstaller 6.x)

The Distribution Dilemma & Solution: Traditional web deployment (e.g. hosting on Vercel or AWS) fails for local hardware monitoring because server-side code measures the cloud VM rather than the visitor's PC. To deliver a real commercial product where any user can monitor their own hardware without installing Python, the application is compiled into a standalone Windows binary using a custom PyInstaller specification.

Component / Stage	Role & Technical Execution
nexus.spec	Custom PyInstaller manifest orchestrating Analysis, Pure Python PYZ, and Binary EXE bundling.
launcher.py	Application entry point. Auto-detects runtime unpack root, initializes Flask, and auto-spawns browser.
sys._MEIPASS Resolver	Dynamic asset locator. When bundled, PyInstaller unpacks templates and static files to a temporary directory; <co
Embedded Python DLL	Bundles <code>python311.dll</code> and native C-extensions (ctypes, select, socket) into the binary.
Virtual File Isolation	Client execution requires zero virtual environments, pip packages, or Python path variables.
Binary Size Efficiency	11.7 MB total binary size achieved by explicitly excluding bloatware (Tkinter, Matplotlib, NumPy, Pandas).
Browser Auto-Trigger	Background daemon executes <code>webbrowser.open('http://127.0.0.1:8050')</code> 2 seconds after server so

3. Backend Architecture & Python Stack

3a. Flask 3.x — Threaded Micro-Framework

Attribute	Implementation Detail
Role	Serves local Single Page Application and powers 18 RESTful micro-endpoints.
Concurrency	Executed in threaded mode (<code>threaded=True</code>), processing simultaneous UI polling without blocking.
Template System	Configured dynamically with custom <code>template_folder</code> and <code>static_folder</code> paths for PyInsta
Zero-Overhead	Replaced Plotly Dash (which required 15MB of client JS and caused 6s bootstrap lags) with sub-millisecond Flask rou

3b. psutil 6.1 — Low-Level OS Telemetry Engine

psutil Subsystem	Metrics Extracted & Handling
psutil.cpu_percent(percpu=T)	Per-core load percentages across all logical threads. Primed twice at startup to establish baseline ticks.
psutil.virtual_memory()	Total, active, cached, and percent RAM consumption.
psutil.swap_memory()	Paging file allocations and commit limits.
psutil.net_io_counters()	Differential network upload and download throughput (bytes/sec) computed via time deltas.
psutil.disk_partitions()	All mounted physical and logical drives (NTFS, FAT32) with storage utilization percentages.
psutil.process_iter()	Scans all live Windows tasks with PID, CPU%, memory footprint, execution status, and owner.
psutil.cpu_freq()	Hardware clock telemetry: current, minimum, and maximum MHz frequencies.
psutil.cpu_count(log=False)	Physical silicon core count, engineered with <code>or 1</code> safety fallback for virtualized environments.
Windows Compatibility	<code>sensors_temperatures()</code> completely omitted due to Windows NT platform incompatibility.

3c. ReportLab 4.2.5 — In-Memory Snapshot Generator

ReportLab Primitive	Industrial Application in NEXUS
SimpleDocTemplate	Generates structured, professional A4 / Letter executive telemetry snapshot documents.
io.BytesIO	Constructs binary PDF documents entirely within system RAM—zero temporary files written to client disk.
Table + TableStyle	Renders stylized CPU core tables, process audit lists, and memory breakdown grids with alternating backgrounds
HRFlowable & Paragraph	Applies brand-aligned color accents (#a855f7, #38bdf8) and typography matching the UI theme.
Dual Usage	Drives the in-app <code>/api/export-pdf-report</code> endpoint and generates offline interview/tech manuals.

3d. Python Standard Library Subsystems

Module	Architectural Responsibility
threading	Runs the background daemon sampler thread continuously without blocking incoming web requests.
threading.Lock	<code>data_lock</code> and <code>proc_lock</code> guarantee thread-safe read/write operations against shared m
collections.deque	Circular memory buffer (<code>maxlen=300</code>) maintaining 5 minutes of sliding telemetry with automatic O(1) ev
statistics	Calculates arithmetic mean (μ) and sample standard deviation (σ) for the real-time Z-score anomaly detecto
csv & io.StringIO	Streams CSV telemetry snapshots directly to HTTP clients via in-memory memory buffers.
json	Serializes metrics into structured JSON payloads for UI rendering and export downloads.
os & platform	Extracts Windows build identifiers, hostnames, and enforces PID protection against self-termination.

4. Frontend Client Stack (Vanilla JS SPA + Landing Page)

4a. UI Dashboard (Single Page Architecture)

Built strictly with **Vanilla JavaScript ES6+ and CSS3**—no React, Vue, or Webpack dependencies. All 9 tabs reside permanently in the DOM; switching tabs is executed via CSS display: none / block toggles in **0.0 milliseconds**. A master interval polls /api/metrics every 1 second, asynchronously updating Canvas charts and DOM nodes.

Layer / Technology	Specifications & Capabilities
HTML5 DOM Structure	9 dedicated tabs: Dashboard, CPU Cores, Memory, Processes, Countermeasures, Twin, Storage, Alerts, Settings.
CSS3 Glassmorphism	Frosted glass styling via <code>backdrop-filter: blur(12px)</code>, custom CSS variables, and full responsive grid.
Chart.js 4.4.2	Locally bundled (205 KB, offline). Renders CPU, RAM, and Network line charts with animations disabled for speed.
Safety Modal System	Intercepts process kills, displaying process name, PID, and requesting explicit user authorization.
Audio Synthesizer	HTML5 Web Audio API beep synthesizer alert when critical CPU thresholds (>90%) are breached.
Multi-Format Exporter	Navbar dropdown & Settings panel export buttons triggering CSV, JSON, and PDF downloads.

4b. Vercel Public Distribution Landing Page (landing/index.html)

A standalone commercial landing page designed for cloud deployment on **Vercel**. It features hero branding, interactive feature cards, tech stack pills, setup walkthroughs, and direct one-click download buttons delivering NEXUS_CPU_Analyzer.exe to visitors worldwide.

5. Project Repository & Deployment File Structure

File / Directory	Size / Lines	Commercial Function
dist/NEXUS_CPU_Analyzer.exe	11.7 MB	The Standalone Commercial Product — zero-dependency Windows desktop binary.
launcher.py	52 lines	Application bootstrapper; manages sys._MEIPASS resolution and browser auto-spawn.
nexus.spec	65 lines	PyInstaller production packaging manifest (data bundling, exclusions, flags).
landing/index.html	240 lines	Public product marketing and binary download hub (hosted on Vercel).
app.py	990 lines	Core Flask engine, low-level psutil telemetry sampler, countermeasures, REST APIs.
templates/index.html	935 lines	Single Page Application dashboard (9 tabs, hardware twin, modals).
static/style.css	1,360 lines	Glassmorphic dark UI theme, animations, report cards, export dropdown.
static/script.js	1,070 lines	Vanilla JS polling engine, Chart.js managers, modal controllers, filters.
static/chart.min.js	205 KB	Chart.js 4.4.2 production bundle (stored locally for 100% offline support).
generate_interview_pdf.py	450 lines	Compiles comprehensive ELI10 + Technical Defense interview manual.
generate_techstack_pdf.py	450 lines	This script — compiles complete technical stack and architecture documentation.
requirements.txt	3 lines	Python package manifest (Flask, psutil, ReportLab) for developers.
vercel.json	22 lines	Vercel serverless routing configuration for cloud landing page.
.gitignore	28 lines	Excludes build caches, personal PDFs, and temporary logs from Git.
START_NEXUS.bat	10 lines	Convenience batch script to run app directly from source in development.

6. Key Architectural Decisions & Trade-Off Matrix

Decision Domain	Selected Strategy	Rejected Alternative	Core Engineering Justification
Product Delivery	Local Executable (.EXE)	Pure SaaS Cloud Host	SaaS measures cloud VM CPU; local binary guarantees true ha

Decision Domain	Selected Strategy	Rejected Alternative	Core Engineering Justification
Packaging Pipeline	PyInstaller 6.x (.spec)	Raw Python Script	Eliminates Python runtime and pip setup prerequisites for end u
Web Micro-Engine	Flask 3.x (Threaded)	Plotly Dash	Eliminated 6s bootstrap delays, 15MB JS bloat, and callback de
Frontend Framework	Vanilla JS (ES6+)	React / Next.js / Vue	Zero build step, zero npm vulnerabilities, 0ms tab transitions, m
Chart Visualization	Chart.js (Local Bundle)	Plotly CDN / D3.js	Full offline independence; lightweight Canvas rendering with ze
Telemetry Storage	In-Memory Deque (300)	SQLite / TimescaleDB	Microsecond memory lookups; zero disk I/O thrashing; automat
Concurrency Model	1 Daemon Thread + Locks	Asyncio / Celery	psutil system calls are synchronous; dedicated thread with mute
Report Generation	ReportLab In-Memory (RAM)	Headless Chrome / Puppeteer	Pure Python compilation; generates full visual PDF reports in R